

Kura

RYO-CHAN Virtual Hackathon 2026 — Project Submission Form

Members

Total Members	1	
Name	Role	Email
Lydia Solomon	Solo Builder / System Architect & Product Designer	lydiasolomon137@gmail.com

Overview

Project	Kura
Track	Track 01: Autonomous Agents
Problem	Autonomous market agents make unverified, non-deterministic decisions on raw tool outputs without pre-trade invariant checks, exposing capital to stale feeds, simulated data, and unconstrained model hallucinations.
Solution	Kura is a deterministic pre-trade execution firewall and black-box cryptographic audit engine. Sits directly between RYO-CHAN's market-intelligence tools and agent execution loops to evaluate five microsecond invariant gates, dynamically clamp allocation via ATR volatility and market breadth, and seal decisions into an append-only SHA-256 hash chain.
Key Features	• Five Deterministic Invariant Gates: FRESHNESS, ORACLE, DATA PROVENANCE, EVIDENCE FLOOR and MARKET CONTEXT, short-circuiting on first failure. Measured 69-140 μs warm, 350-647 μs cold. • Volatility-Adjusted Allocation: Dynamic risk clamping via ATR(14), then scaled by market breadth; 0% capital at hyper-volatility (>50% ATR). • Genuine Rate-Limit Handling: HTTP 429 backoff with server Retry-After parsing and full jitter. • Append-Only Cryptographic Audit Ledger: Independent Python and TypeScript hash-chain verifiers. • Calm Multi-Screen Interface: Dedicated routes for Evaluator, Chaos Lab, Audit Ledger, and Gateway Docs.
Target Users	Autonomous trading agent developers, algorithmic risk teams, and Web3 agent protocols requiring deterministic execution bounds under Rule 6.05.
Scope	Fullstack monorepo with an MCP interceptor daemon, deterministic arbiter reading both analyze_token and market_overview, authentic 6-tool JSON-RPC conformance peer, adversarial chaos simulation lab, and a responsive web operations console.
Limitations	Read-only execution firewall; per Rule 6.05, it does not sign transactions, manage private keys, or route live orders to DEXs/CEXs. Live upstream signals depend on RYO-CHAN MCP API availability.

Tech Stack

Frontend Stack	Next.js 14 (App Router), React 18, Tailwind CSS, Server-Sent Events (SSE) telemetry client.
Backend Stack	Node.js, TypeScript, Fastify MCP Gateway, SQLite (better-sqlite3) with WAL-mode hash chain, Zod schema validation.
AI Model(s)	None in the critical execution path (deliberate zero-hallucination architectural posture); Claude Code utilized strictly as developer tooling.
Other Technologies	Vitest (153 automated tests across 10 suites), Python 3 (independent SHA-256 cryptographic audit verifier), Docker.

Repository / Demo

Github Repository	https://github.com/RYO-Digital/ryochan-hackathon_repository-249 Mirror: https://github.com/Lideeyah/kura
Demo Video	https://youtu.be/1MMqfDH9mUA Offline mirror (silent, in repo): kura-demo-walkthrough.mp4
Documentation	README.md, SUBMISSION.md, and /app/integration interactive developer guide in the repo.

Testing Information

How to run the Project	Run <code>npm run dev</code> to launch the deterministic engine on :4000 and the web console on :3200.
Prerequisites	Node.js v20+ LTS, npm v9+, Python 3.9+ (for external hash-chain audit script).
Installation Steps	<pre>git clone https://github.com/RYO-Digital/ryochan-hackathon_repository-249.git cd ryochan-hackathon_repository-249 npm install cp .env.example .env</pre>
Environment Variables	<pre>RYO_MCP_URL=https://app-ryochan.com/api/mcp RYO_MCP_KEY=<provided_by_organizer> RYO_MCP_TRANSPORT=stdio (default) or http (live) PORT=4000</pre>
Build Command	<code>npm run build</code>
Run Command	<code>npm run dev</code>
Test Command	<code>npm test</code>
Test Account(s)	None required; the system runs locally out-of-the-box against the zero-credential 6-tool conformance peer. Run <code>python3 skills/verify_provenance/tool.py --all</code> for external proof verification.